

1. Business Problem
2. Architecture
3. End-to-End Workflow
4. Components Used
5. Why Each Technology Was Chosen
6. Challenges
7. Production Considerations
8. Interview Questions & Answers

The following explanation is based on the projects listed in your resume.

PROJECT 1

SANOFI – AIOps Incident Resolution Platform

Domain

Healthcare IT Operations

Duration

Dec 2023 – Apr 2025

Problem Statement

Sanofi had thousands of alerts generated daily from:

- Splunk
- ServiceNow
- Control-M
- Windows Servers
- Linux Servers

L1 engineers manually:

1. Opened ticket
2. Checked historical incidents
3. Searched SOP documents

4. Looked for remediation steps
5. Escalated if unable to resolve

This process was slow.

Average Resolution Time (MTTR):

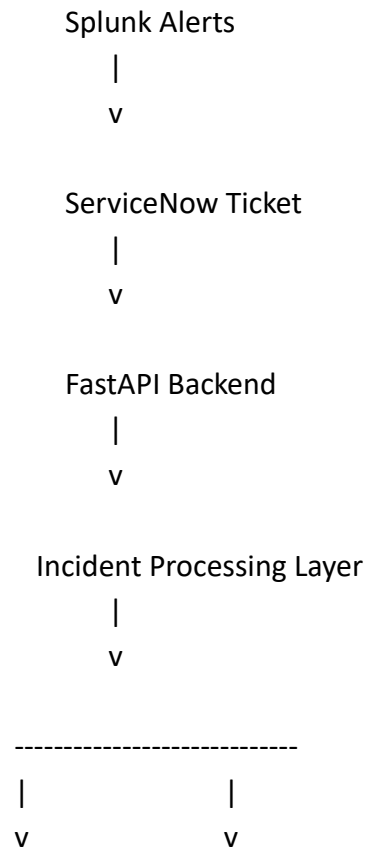
- 30–45 minutes

Goal:

Build an AI Assistant that:

- Understands incident
- Finds similar historical incidents
- Retrieves SOPs
- Suggests remediation steps
- Reduces MTTR

High Level Architecture



Embedding Model
(Titan Embeddings)

GPT-4 LLM

|
v

FAISS Vector DB <-----

|
v

Historical Incidents

SOP Documents

Knowledge Articles

Runbooks

|
v

Retrieved Context

|
v

GPT-4 Prompt

|
v

AI Recommendation

|
v

L1 Engineer Review

|
v

Resolve / Escalate

Data Sources

Splunk

Contains:

CPU Utilization Alert

Server: App-Server-101

CPU Usage: 98%

ServiceNow

Contains:

Incident Number

Priority

Assigned Group

Status

Control-M

Contains:

Job Failures

Batch Processing Errors

Dependency Failures

SOP Documents

Contains:

How to restart service

How to clean disk

How to increase memory

Step 1: Data Ingestion

Daily batch process:

Historical Incidents

SOP Documents

Knowledge Articles

Runbooks

loaded from:

ServiceNow

Splunk

Control-M

Step 2: Document Chunking

Example SOP:

Restart Apache Service

Step 1:

Login Server

Step 2:

Run systemctl restart apache2

Chunked into:

Chunk 1

Restart Apache Service

Step 1 Login Server

Step 2 Run systemctl restart apache2

Why Chunking?

GPT cannot search entire documents efficiently.

Chunking improves:

- Retrieval

- Accuracy
 - Token usage
-

Step 3: Embedding Generation

Used:

Amazon Titan Embeddings

Convert text into vectors.

Example:

"CPU utilization high"

→

[0.21,0.45,0.88,0.17]

Step 4: Store in FAISS

Chunk

Vector

Metadata

Stored in:

FAISS

Example:

```
{  
  "ticket_id":"INC1001",  
  "category":"CPU",  
  "resolution":"Restart Apache"  
}
```

Why FAISS?

Interview Answer:

Since knowledge volume was not extremely large and latency was important, FAISS provided fast local vector search without requiring managed infrastructure.

Runtime Flow

User Query

Incident:

CPU Usage 95%

Server APP001

Embedding Search

Convert incident into vector.

Search:

Top K = 5

inside FAISS.

Retrieve:

Historical incidents

SOPs

Runbooks

Context Building

Example:

Incident:

CPU 95%

Similar Incident:

CPU spike caused by Apache process

Resolution:

Restart Apache Service

Prompt Construction

You are an AIOps Engineer.

Incident:
{incident}

Context:
{retrieved_docs}

Suggest resolution.

GPT-4 Response

Root Cause:
Apache process consuming CPU

Recommended Action:
Restart Apache service

Confidence:
92%

Human-in-the-loop

Recommendation is never auto-executed.

L1 Engineer validates.

Approve
Reject
Escalate

Security

Implemented:

IAM

Role-based access

Encryption

S3 encryption

Audit Logs

Track:

Who queried

What recommendation generated

Evaluation Metrics

Retrieval Accuracy

Relevant Chunks Retrieved

Total Retrieved Chunks

Precision

Correct Recommendations

Total Recommendations

Hallucination Rate

Unsupported Responses

Total Responses

Production Challenges

Challenge 1

Different incident descriptions for same issue.

Example:

CPU Spike

Processor High

CPU Utilization Alert

Solution:

Semantic Search using Embeddings.

Challenge 2

Hallucinations

Solution:

RAG + Context Grounding.

Challenge 3

Large SOP Documents

Solution:

Chunking with overlap.

PROJECT 2

CITIZENS FINANCIAL GROUP

Agentic Underwriting Assistant

Domain

Banking

Duration

May 2025 – Mar 2026

Business Problem

Loan approval process required multiple teams.

Manual checks:

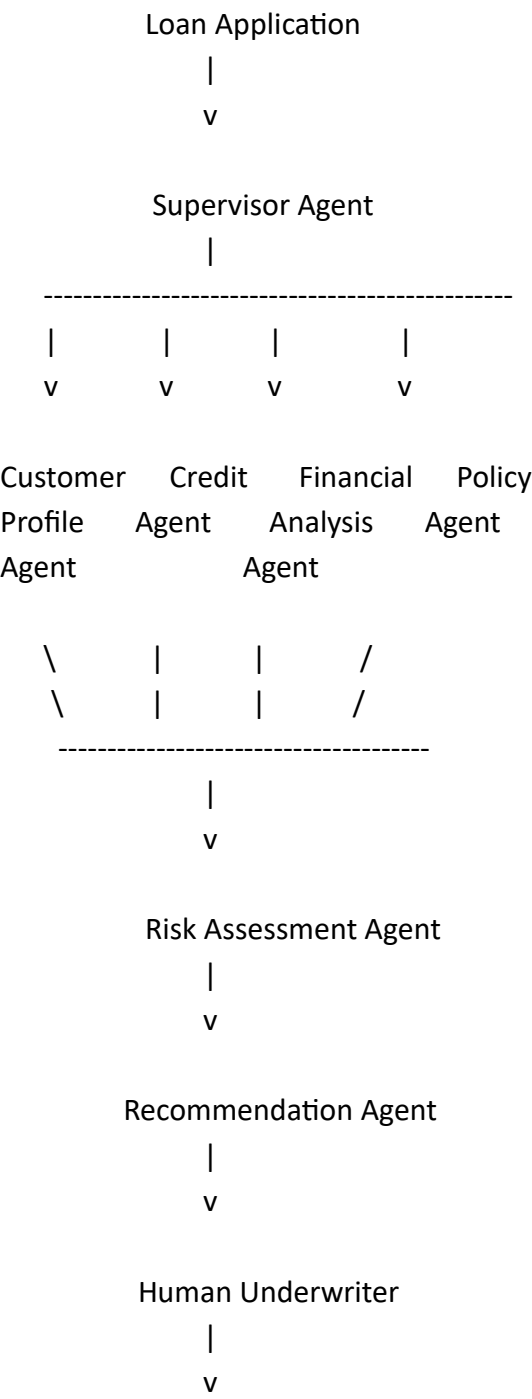
1. Customer Profile
2. Credit Score
3. Income Verification
4. Policy Validation
5. Risk Assessment
6. Recommendation

Took hours.

Goal:

Build Agentic AI system that assists underwriters.

High Level Architecture



Approve / Reject Loan

Why Agentic AI?

Single LLM cannot perform all tasks efficiently.

Different agents specialize in:

- Credit analysis
- Income verification
- Policy validation

Improves reasoning.

MCP Integration Layer

Agents need external data.

MCP Server provides:

Customer Database

Credit Bureau

Financial Documents

Policy Documents

Architecture:

Agent

|

v

MCP Client

|

v

MCP Server

|

Enterprise Systems

Agent Responsibilities

Customer Profile Agent

Reads:

Name

Age

Employment

Loan Amount

Produces:

Customer Summary

Credit Assessment Agent

Checks:

Credit Score

Defaults

Delinquencies

Produces:

Credit Risk

Financial Analysis Agent

Reads:

Payslips

Bank Statements

Tax Returns

Calculates:

DTI Ratio

Income Stability

Cash Flow

Policy Validation Agent

Reads:

Bank Policy Documents

Checks:

Eligibility

Example:

Minimum Credit Score = 700

Risk Assessment Agent

Combines outputs.

Generates:

Risk Score

Recommendation Agent

Produces:

Approve

Reject

Conditional Approval

RAG Layer

Documents:

Underwriting Policies

Risk Rules

Historical Loans

Indexing Flow

Policies

Historical Loans

Risk Guidelines

↓

Chunking

↓

Titan Embeddings

↓

Pinecone

Why Pinecone?

Interview Answer:

Pinecone was chosen because underwriting knowledge continuously grows and Pinecone provides managed scalable vector search with low latency and high availability.

Runtime Flow

Loan Request:

Customer:

John

Credit Score:

760

Income:
₹15 Lakhs

Loan:
₹50 Lakhs

Supervisor Agent

Creates workflow.

Step 1:
Profile Agent

Step 2:
Credit Agent

Step 3:
Financial Agent

Step 4:
Policy Agent

Agent Outputs

Profile Agent:
Stable Employment

Credit Agent:
Excellent Credit

Financial Agent:
Low DTI Ratio

Policy Agent:
Eligible

Risk Assessment Agent

Combines outputs:

Risk Score: Low

Recommendation Agent

Recommendation:

Approve Loan

Confidence:

94%

Human-in-the-loop

Final decision always made by underwriter.

AI Suggestion



Human Review



Approval

Security

Banking domain requires:

IAM

Access control

Encryption

Data encrypted

Audit Logging

Every recommendation logged

PII Protection

Mask:

PAN

SSN

Account Number

LangGraph Flow

START



Supervisor Agent



Profile Agent



Credit Agent



Financial Agent



Policy Agent



Risk Agent



Recommendation Agent



END

Why LangGraph Instead of LangChain?

Interview Answer:

LangChain works well for linear workflows. However, underwriting requires multiple agents collaborating, maintaining state, and making conditional decisions. LangGraph provides stateful orchestration and agent coordination, making it more suitable for complex decision-making workflows.

2-Minute Interview Summary

Sanofi

I developed a GenAI-powered AIOps Incident Resolution Platform for L1 support teams. The system used a RAG architecture where historical incidents, SOPs, and knowledge articles were embedded using Amazon Titan Embeddings and stored in FAISS. When a new incident arrived from Splunk or ServiceNow, relevant documents were retrieved and provided to GPT-4, which generated remediation recommendations. FastAPI services were deployed on AWS, and all recommendations went through human validation before resolution or escalation.

Citizens Financial Group

I developed an Agentic AI Underwriting Assistant using LangGraph, AWS Bedrock, Claude 3.5 Sonnet, MCP, Pinecone, and Amazon Titan Embeddings. The platform consisted of multiple specialized agents for customer profiling, credit assessment, financial analysis, policy validation, risk assessment, and recommendation generation. Agentic RAG was used to retrieve underwriting guidelines and historical lending knowledge. Final recommendations were reviewed by underwriters before approval, ensuring compliance, governance, and explainability.

The interviewer will likely test:

1. Real project ownership
2. RAG implementation details
3. Agentic AI concepts
4. Production deployment

5. AWS architecture
 6. Evaluation & Monitoring
 7. LLM fundamentals
 8. Design decisions and tradeoffs
 9. Multi-agent architecture
 10. Scalability
-

SANOFI PROJECT QUESTIONS (VERY HIGH PROBABILITY)

1. Can you explain the Sanofi AIOps Incident Resolution Platform end-to-end?

Answer

"In the Sanofi project, the objective was to reduce incident resolution time for L1 support teams.

The platform ingested historical incident tickets from ServiceNow, monitoring alerts from Splunk, Control-M job failure logs, SOP documents, and knowledge base articles.

We built a RAG-based system using LangChain, GPT-4, Amazon Titan Embeddings, and FAISS.

Whenever an L1 engineer received a new alert, the incident details were sent to our FastAPI service.

The query was converted into embeddings using Titan Embeddings and searched against the FAISS vector store.

The most relevant historical incidents, SOPs, and remediation documents were retrieved and passed to GPT-4.

The LLM generated probable root causes, recommended remediation steps, confidence score, and escalation suggestions.

The recommendation was reviewed by the L1 engineer before execution, ensuring a human-in-the-loop workflow.

The solution significantly reduced investigation time by providing contextual incident resolution guidance."

2. What was the source of data?

Answer

"The data came from multiple enterprise operational systems.

ServiceNow provided incident tickets and resolution notes.

Splunk provided monitoring alerts, application logs, and infrastructure events.

Control-M provided batch job execution logs and failure information.

Additionally, SOP documents and knowledge base articles were stored in S3 and indexed for retrieval.

These sources formed the knowledge repository for the RAG pipeline."

3. Explain the RAG pipeline in detail.

Answer

"First, documents were collected from ServiceNow, Splunk, Control-M, and knowledge repositories.

The documents were cleaned and normalized.

We applied recursive chunking with overlap to preserve context.

Chunks were converted into embeddings using Amazon Titan Embeddings.

The embeddings were stored in FAISS.

When a new incident arrived, the incident description was embedded and searched against FAISS using similarity search.

Top-k relevant documents were retrieved.

The retrieved context along with the incident details was passed to GPT-4 through a structured prompt.

GPT-4 generated root cause analysis and remediation recommendations.

The final response was shown to the L1 analyst for validation."

4. Why FAISS and not Pinecone?

Answer

"FAISS was selected because the dataset size was manageable and deployed within our AWS environment.

FAISS provided low latency retrieval and reduced operational costs.

For the banking project, Pinecone was preferred because it required scalable managed vector infrastructure for larger datasets and multiple concurrent users."

5. Why use GPT-4?

Answer

"GPT-4 provided better reasoning capability for root cause analysis.

The model was capable of understanding incident patterns, correlating logs with historical incidents, and generating detailed remediation guidance.

We observed higher response quality compared to smaller models."

6. How did you reduce hallucinations?

Answer

"I used multiple techniques.

First, RAG grounding ensured responses were generated only from retrieved documents.

Second, prompts explicitly instructed the model not to answer outside the provided context.

Third, confidence thresholds were used.

Fourth, human validation was mandatory before execution.

Finally, evaluation datasets were used to continuously measure hallucination rates."

7. What chunking strategy did you use?

Answer

"I used Recursive Character Text Splitting with overlap.

The chunk size was around 500 to 1000 tokens with approximately 100 token overlap.

This preserved contextual continuity while maintaining retrieval efficiency."

8. What metadata was stored in FAISS?

Answer

"Each vector contained metadata such as incident ID, source system, severity level, application name, timestamp, team ownership, and document type."

This allowed filtered retrieval and improved relevance."

9. How was evaluation performed?

Answer

"We created a golden dataset consisting of historical incidents and known resolutions.

The generated recommendations were compared against actual resolutions.

Metrics included retrieval relevance, response accuracy, hallucination rate, and user acceptance rate."

10. How did human-in-the-loop work?

Answer

"L1 engineers reviewed every recommendation before execution.

The AI acted as a decision-support system rather than an autonomous resolver.

If the recommendation was approved, the analyst proceeded with remediation.

Otherwise, the ticket was escalated to L2 or L3 teams."

AGENTIC AI QUESTIONS

11. Difference between LangChain and LangGraph?

Answer

"LangChain is primarily used for sequential workflows such as RAG pipelines and prompt orchestration.

LangGraph is designed for stateful multi-agent systems where agents collaborate, maintain memory, and make dynamic decisions.

In my banking project, LangGraph was used because multiple agents had to coordinate underwriting tasks.

In Sanofi, LangChain was sufficient because the workflow was mostly linear."

12. When would you use Agentic AI instead of RAG?

Answer

"If the problem requires planning, reasoning, tool usage, and multiple decision steps, I would use Agentic AI.

If the problem only requires knowledge retrieval and question answering, RAG is sufficient."

MCP QUESTIONS

13. Explain MCP.

Answer

"Model Context Protocol is a standard mechanism that allows AI agents to securely interact with external systems.

It standardizes tool discovery, authentication, and data access.

Instead of creating custom integrations for every system, MCP provides a reusable protocol for agent communication."

14. Why MCP instead of APIs?

Answer

"Traditional APIs require custom integration logic for every application.

MCP provides a standardized interface that allows AI agents to discover and invoke tools dynamically.

This reduces integration complexity and improves scalability."

AWS QUESTIONS

15. Explain your AWS architecture.

Answer

"S3 stored documents and operational knowledge.

Titan Embeddings generated vector representations.

FAISS stored embeddings.

FastAPI services ran on EC2.

Lambda functions handled ingestion and preprocessing.

CloudWatch monitored application health.

IAM managed secure access.

GPT-4 generated recommendations after retrieval."

16. Why Lambda?

Answer

"Lambda was used for event-driven ingestion and preprocessing tasks.

Whenever new documents arrived, Lambda automatically processed and indexed them."

DEPLOYMENT QUESTIONS

17. How would you deploy a multi-agent application?

Answer

"I would containerize the application using Docker.

FastAPI services would expose agent endpoints.

Deployment can be performed on ECS, EKS, or Kubernetes.

Vector databases would be hosted separately.

Monitoring would be handled through CloudWatch and application logs.

CI/CD pipelines would automate deployment."

LLM QUESTIONS (HIGH CHANCE)

18. What is temperature?

Answer

"Temperature controls randomness.

Lower values produce deterministic responses.

Higher values increase creativity.

For enterprise applications, I usually keep temperature between 0 and 0.3."

19. What is Top-K?

Answer

"Top-K limits token selection to the K most probable tokens before sampling."

20. What is Top-P?

Answer

"Top-P selects tokens from the smallest cumulative probability distribution that exceeds a threshold."

21. What is an embedding?

Answer

"An embedding is a dense vector representation of text that captures semantic meaning. Similar texts generate vectors that are close in vector space."

QUESTIONS FROM THE MOVATE JD

The JD specifically emphasizes Agentic AI, Multi-Agent Systems, LLMOps, Observability, and Production AI Systems.

Expect these:

Explain observability in LLM applications.

"Observability involves tracking prompts, responses, latency, token usage, retrieval quality, failures, hallucinations, and user feedback to ensure production reliability."

How would you handle 10,000 concurrent users?

"I would horizontally scale FastAPI services, use load balancers, cache frequent queries, move FAISS to a distributed vector database, and implement asynchronous processing."

What is LLMOps?

"LLMOps is the practice of managing the lifecycle of LLM applications including deployment, monitoring, evaluation, governance, security, and continuous improvement."

How do you secure a GenAI application?

"I implement IAM-based access control, encryption at rest and in transit, audit logging, PII masking, input validation, and human approval workflows."

Explain semantic search.

"Semantic search retrieves information based on meaning rather than exact keyword matching using embeddings and vector similarity."

If this is a true L2 technical round, I would prioritize mastering:

1. Banking Architecture (already asked in L1)
2. Sanofi End-to-End Architecture
3. RAG Pipeline Deep Dive
4. LangGraph vs LangChain
5. MCP Deep Dive
6. FAISS vs Pinecone
7. Hallucination Reduction
8. Evaluation Metrics
9. AWS Deployment Architecture
10. Multi-Agent System Design

These are the areas where interviewers usually keep drilling until they find the limit of your practical knowledge.

These are the **follow-up answers that interviewers usually ask after your first answer**. I'll make them project-specific to your **Banking (Agentic AI)** and **Sanofi (RAG)** projects.

Explain End-to-End RAG Architecture

Answer

"In the Sanofi project, the RAG architecture consisted of an ingestion layer, embedding layer, vector storage layer, retrieval layer, and generation layer.

Data was collected from ServiceNow incidents, Splunk alerts, Control-M logs, SOP documents, and knowledge articles.

The documents were cleaned, chunked, and converted into embeddings using Amazon Titan Embeddings.

The embeddings were stored in FAISS.

When a new incident arrived, the incident description was converted into an embedding and similarity search was performed.

The top relevant chunks were retrieved and passed along with the user query to GPT-4.

GPT-4 generated the root cause analysis and remediation recommendation.

The recommendation was finally reviewed by the L1 analyst before execution."

Why Are Embeddings Required?

Answer

"Embeddings convert text into numerical vectors while preserving semantic meaning.

Without embeddings, we would have to rely on keyword matching.

Embeddings allow the system to retrieve information based on meaning rather than exact words."

How Do You Decide Chunk Size?

Answer

"I usually start with chunks between 500 and 1000 tokens and overlap between 50 and 150 tokens.

If the document contains procedures or SOPs, larger chunks work better.

If the document contains FAQs or short sections, smaller chunks improve retrieval precision.

The final choice depends on retrieval evaluation results."

Challenges in Semantic Search

Answer

"I faced issues such as similar concepts returning irrelevant documents, duplicated chunks, incomplete context, and domain-specific terminology.

To improve results, I used metadata filtering, better chunking strategies, and retrieval evaluation."

Improve Retrieval Accuracy

Answer

"I would improve chunking, use better embedding models, optimize metadata filtering, apply reranking models, tune Top-K retrieval, and evaluate retrieval quality using golden datasets."

Keyword Search vs Vector Search

Answer

"Keyword search matches exact words.

Vector search matches semantic meaning.

For example, 'loan default risk' and 'credit repayment failure' may not share keywords but vector search can identify them as related."

How Are Embeddings Stored In Pinecone?

Answer

"Each chunk is converted into a vector using Titan Embeddings.

The vector, chunk text, and metadata are stored together in Pinecone.

When a query arrives, Pinecone compares the query embedding against stored vectors and returns the most similar chunks."

Similarity Metrics

Answer

"The most common similarity metrics are Cosine Similarity, Dot Product, and Euclidean Distance.

In most semantic search applications, Cosine Similarity is commonly used."

Handling Duplicate Embeddings

Answer

"I maintain unique document IDs and version numbers.

When a document changes, I re-embed only the modified document and update the corresponding vectors.

Old vectors are deleted to avoid stale information."

Role of LangChain in Your Project

Answer

"In the Sanofi project, LangChain was used to orchestrate the RAG workflow.

It handled document loading, chunking, embedding generation, retrieval, prompt construction, and GPT-4 integration."

Chains vs Agents

Answer

"A Chain follows a predefined sequence of steps.

An Agent dynamically decides what actions to take.

The Sanofi project primarily used chains.

The banking project used agents through LangGraph."

LangChain Advantages Over Direct LLM API

Answer

"LangChain provides built-in support for RAG, memory, vector databases, prompt templates, tools, and workflow orchestration.

Without LangChain, I would need to manually implement all those components."

Conversation Memory

Answer

"I can use Conversation Buffer Memory, Summary Memory, or Vector Memory depending on the use case.

For long conversations, Summary Memory is usually more efficient."

LLM Response Evaluation

Answer

"I evaluate response accuracy, relevance, faithfulness, completeness, hallucination rate, latency, and user acceptance rate."

Metrics To Compare Prompts

Answer

"I compare answer relevance, factual accuracy, hallucination rate, token usage, latency, and user feedback scores."

Detect Model Drift

Answer

"I continuously evaluate model outputs against a golden dataset.

If retrieval accuracy or answer quality degrades over time, it indicates potential drift."

Prompt Engineering Improvement Example

Answer

"In the Sanofi project, adding explicit instructions such as 'Answer only from retrieved context' significantly reduced hallucinations and improved response consistency."

Agent vs Workflow

Answer

"A workflow follows fixed predefined steps.

An agent can make decisions dynamically based on context and goals."

Multi-Agent Architecture

Answer

"In the banking project, I used a Supervisor Agent coordinating Customer Profile, Credit Assessment, Financial Analysis, Policy Validation, Risk Assessment, and Recommendation Agents.

Each agent handled a specific responsibility and returned results to the Supervisor Agent."

How Agents Communicate

Answer

"In LangGraph, agents communicate through a shared state object.

Each agent updates the state and passes it to the next agent."

Planning and Memory in Agentic AI

Answer

"Planning helps agents determine the sequence of actions.

Memory allows agents to retain previous context and decisions throughout the workflow."

AWS Deployment Architecture

Answer

"S3 stored documents.

Lambda handled ingestion events.

Titan Embeddings generated vectors.

Pinecone stored embeddings.

FastAPI services ran on EC2.

AWS Bedrock provided Claude access.

CloudWatch handled monitoring."

API Gateway Purpose

Answer

"API Gateway acts as the entry point.

It handles authentication, request routing, throttling, and API management."

Why FastAPI?

Answer

"FastAPI provides high performance, asynchronous processing, automatic API documentation, and strong Pydantic integration."

Authentication In FastAPI

Answer

"I generally use JWT tokens, OAuth2, API keys, and IAM-based access control."

High Concurrency Optimization

Answer

"I use async endpoints, connection pooling, caching, horizontal scaling, load balancers, and background task processing."

Factors Affecting Embedding Quality

Answer

"Embedding model quality, chunking strategy, document preprocessing, domain-specific terminology, and metadata quality significantly impact retrieval performance."

Can Embeddings Be Regenerated?

Answer

"Yes, but if a different embedding model is used, all stored vectors should ideally be regenerated to maintain consistency."

API Failure Handling

Answer

"I implement retries with exponential backoff, circuit breakers, fallback responses, timeout controls, and monitoring."

Reduce Token Cost

Answer

"I optimize prompts, reduce unnecessary context, use chunk filtering, caching, response compression, and smaller models where appropriate."

Secure API Keys

Answer

"I store them in AWS Secrets Manager or environment variables and never hardcode them in source code."

Design Policy Document Chatbot

Answer

"I would ingest policy documents into S3, chunk them, generate embeddings, store them in Pinecone, retrieve relevant chunks using RAG, and use an LLM to generate grounded responses."

Insurance Claim Denial Solution

Answer

"I would build a RAG system using historical claim denials, policy rules, and compliance documents."

The system would retrieve similar denial cases and suggest corrective actions with supporting evidence."

Irrelevant Answers In RAG

Answer

"I would check document quality, chunking, embedding model, metadata filters, retrieval scores, prompt design, and reranking effectiveness."

Same Prompt Producing Different Answers

Answer

"I would check temperature settings, retrieved context differences, prompt variations, model version changes, and retrieval consistency."

Scalable System For Millions Of Documents

Answer

"I would use event-driven ingestion with S3 and Lambda, distributed processing, managed vector databases like Pinecone, autoscaling FastAPI services, Bedrock for inference, caching layers, monitoring, and CI/CD pipelines."

What Is Scalability?

Answer

"Scalability means the system can handle increasing users, documents, requests, and workloads without significant performance degradation."

What Is Normalization?

Answer

"In machine learning, normalization scales numerical values into a common range."

In embeddings, vector normalization converts vectors to unit length so similarity calculations such as cosine similarity become more accurate."

Claude LLM - Why Did You Use It?

Answer

"In the banking project, Claude 3.5 Sonnet provided strong reasoning, document understanding, summarization, and structured response generation capabilities, making it suitable for underwriting workflows."

Most Likely L2 Closing Question

"Why should we hire you?"

Answer

"I have hands-on experience building both RAG and Agentic AI systems across banking and healthcare domains. I have worked with LangChain, LangGraph, AWS Bedrock, Pinecone, FAISS, FastAPI, and MCP integrations. Beyond building PoCs, I understand retrieval,

orchestration, evaluation, deployment, observability, and production challenges, which allows me to contribute across the complete GenAI application lifecycle."

If you can confidently answer these plus the Banking-project questions from the previous message, you'll be prepared for most L2 technical discussions around your resume.

. RAG (Retrieval-Augmented Generation) Explain the end-to-end architecture of a RAG application. Why are embeddings required in a RAG pipeline? How do you decide the chunk size and overlap while processing documents? What challenges have you faced with semantic search? How would you improve retrieval accuracy in a RAG application? What is the difference between keyword search and vector search? 2. Vector Database Why did you choose Pinecone over other vector databases? How are embeddings stored and retrieved in Pinecone? What similarity metrics are commonly used in vector search? How would you handle duplicate or outdated embeddings? 3. LangChain Explain the role of LangChain in your project. What are Chains and Agents in LangChain? How do you integrate LangChain with vector databases and LLMs? What are the advantages of LangChain over directly calling an LLM API? How do you maintain conversation memory in LangChain? 4. Prompt Engineering & LLM Evaluation What techniques do you use to reduce hallucinations? How do you evaluate the quality of LLM responses? What metrics can be used to compare two prompts or two models? How would you detect model drift over time? Explain a scenario where prompt engineering significantly improved output quality. 5. Agentic AI What is an AI Agent? What is the difference between an Agent and a Workflow? Have you worked with multi-agent systems? Explain the architecture. How do agents communicate with each other? What is the role of planning and memory in Agentic AI? 6. AWS & Deployment Explain how your GenAI application was deployed on AWS. What is the purpose of API Gateway, Lambda, and S3 in your solution? Where would you host a FastAPI application in AWS? What is the difference between EC2, ECS, EKS, and Lambda? When would you choose Lambda over EC2? 7. FastAPI Why did you choose FastAPI for your application? How do you expose an LLM service using FastAPI? How do you handle authentication and rate limiting in FastAPI? How would you optimize FastAPI for high concurrency? 8. Embeddings What are embeddings and why are they important? Which embedding models have you used? What factors influence embedding quality? Can embeddings be regenerated without affecting retrieval quality? 9. LLM Integration How do you integrate OpenAI or other LLM APIs into an application? How do you handle API failures and retries? What techniques can reduce token usage and cost? How do you ensure secure handling of API keys? 10. Scenario-Based Questions Design a chatbot that answers questions from company policy documents using RAG. How would you build an AI system to analyze insurance claim denials and suggest corrective actions? If users complain about irrelevant answers from a RAG system, how would you troubleshoot the issue? An LLM is producing inconsistent responses for the same prompt. How would you investigate and resolve the problem? Design a scalable GenAI application capable of processing millions of documents daily. I need answers for these questions

These answers are optimized for a **2–4 year GenAI Engineer L2 interview**. Speak naturally and confidently.

1. RAG (Retrieval-Augmented Generation)

Explain the end-to-end architecture of a RAG application.

Answer:

"In my Sanofi project, the RAG pipeline started with document ingestion from ServiceNow incidents, Splunk alerts, Control-M logs, SOPs, and knowledge articles.

The documents were cleaned, chunked, and converted into embeddings using Amazon Titan Embeddings.

These embeddings were stored in FAISS.

When a user submitted a query, the query was also converted into an embedding and semantic similarity search was performed.

The top relevant chunks were retrieved and sent along with the query to GPT-4.

The model generated an answer using only the retrieved context, which helped reduce hallucinations."

Why are embeddings required in a RAG pipeline?

Answer:

"Embeddings convert text into numerical vectors while preserving semantic meaning.

This allows us to retrieve documents based on meaning rather than exact keywords.

Without embeddings, semantic search would not be possible."

How do you decide chunk size and overlap?

Answer:

"I usually start with 500–1000 token chunks and 50–150 token overlap.

For SOPs and policy documents, larger chunks preserve context.

For FAQs or short documents, smaller chunks improve retrieval precision.

The final decision is based on retrieval evaluation results."

What challenges have you faced with semantic search?

Answer:

"I have faced issues such as irrelevant retrieval, duplicate chunks, insufficient context, and domain-specific terminology.

I solved them using metadata filtering, better chunking strategies, and retrieval evaluation."

How would you improve retrieval accuracy?

Answer:

"I would improve chunking, use better embedding models, apply metadata filters, tune Top-K retrieval, use rerankers, and continuously evaluate retrieval performance using golden datasets."

Difference between keyword search and vector search?

Answer:

"Keyword search matches exact words.

Vector search matches semantic meaning.

For example, 'loan default risk' and 'credit repayment failure' may return similar results in vector search even though the keywords are different."

2. Vector Database

Why Pinecone?

Answer:

"In the banking project, I chose Pinecone because it is fully managed, supports automatic scaling, metadata filtering, high availability, and requires minimal operational overhead."

How are embeddings stored and retrieved?

Answer:

"Each chunk is converted into an embedding vector.

The vector, chunk text, and metadata are stored together in Pinecone.

During retrieval, the query is embedded and Pinecone returns the most similar vectors based on similarity scores."

Similarity metrics?

Answer:

"The most common similarity metrics are Cosine Similarity, Dot Product Similarity, and Euclidean Distance.

Cosine Similarity is most commonly used in semantic search."

Handling duplicate or outdated embeddings?

Answer:

"I use unique document IDs and versioning.

When documents are updated, old embeddings are removed and new embeddings are generated and indexed."

3. LangChain

Role of LangChain?

Answer:

"I used LangChain to orchestrate document loading, chunking, embedding generation, vector search, prompt construction, and LLM integration."

Chains vs Agents?

Answer:

"A Chain follows a fixed sequence of steps.

An Agent dynamically decides which actions or tools to use based on the task."

Integration with Vector DB and LLM?

Answer:

"LangChain provides built-in retrievers and vector store integrations.

Retrieved context is passed into prompt templates and then sent to the LLM."

Advantages over direct LLM API?

Answer:

"LangChain simplifies prompt management, RAG implementation, memory handling, tool integration, and workflow orchestration."

Conversation memory?

Answer:

"I can use Conversation Buffer Memory, Summary Memory, or Vector Store Memory depending on the use case."

4. Prompt Engineering & LLM Evaluation

Reduce hallucinations?

Answer:

"I use RAG grounding, strict prompts, source citations, output validation, confidence scoring, and human review workflows."

Evaluate LLM quality?

Answer:

"I evaluate accuracy, relevance, faithfulness, completeness, hallucination rate, latency, and user feedback."

Compare prompts or models?

Answer:

"I compare answer quality, relevance, factual accuracy, token usage, latency, and hallucination rates."

Detect model drift?

Answer:

"I continuously test the model against a golden dataset and monitor changes in accuracy, relevance, and user satisfaction."

Prompt engineering improvement example?

Answer:

"In the Sanofi project, explicitly instructing GPT-4 to answer only from retrieved context significantly reduced hallucinations."

5. Agentic AI

What is an AI Agent?

Answer:

"An AI Agent is an autonomous system that uses an LLM for reasoning and can interact with tools, APIs, databases, or external systems to complete tasks."

Agent vs Workflow?

Answer:

"A workflow follows predefined steps."

An agent dynamically decides what action to take next based on the current situation."

Multi-agent architecture?

Answer:

"In the banking project, I implemented Customer Profile, Credit Assessment, Financial Analysis, Policy Validation, Risk Assessment, and Recommendation Agents coordinated by a Supervisor Agent."

How do agents communicate?

Answer:

"In LangGraph, agents communicate through a shared state object that is updated and passed between nodes."

Planning and memory?

Answer:

"Planning helps agents determine the sequence of actions.

Memory helps them retain context and previous decisions."

6. AWS & Deployment

Deployment architecture?

Answer:

"S3 stored documents.

Lambda handled ingestion.

Titan Embeddings generated vectors.

Pinecone stored embeddings.

FastAPI ran on EC2.

Bedrock provided Claude access.

CloudWatch handled monitoring."

Purpose of API Gateway, Lambda, and S3?

Answer:

"API Gateway exposes APIs securely.

Lambda handles event-driven processing.

S3 stores documents and data."

Where host FastAPI?

Answer:

"I would host FastAPI on EC2, ECS, or EKS depending on scalability requirements."

EC2 vs ECS vs EKS vs Lambda?

Answer:

EC2: Virtual machines.

ECS: Container orchestration managed by AWS.

EKS: Kubernetes managed by AWS.

Lambda: Serverless execution.

Lambda vs EC2?

Answer:

"I choose Lambda for event-driven workloads and short-running tasks.

I choose EC2 for long-running applications and full environment control."

7. FastAPI

Why FastAPI?

Answer:

"It is lightweight, high-performance, supports async programming, automatic API documentation, and integrates well with Pydantic."

Expose LLM service?

Answer:

"I create REST endpoints in FastAPI that receive user queries, invoke retrieval and LLM workflows, and return responses."

Authentication and rate limiting?

Answer:

"I use JWT tokens, OAuth2, API keys, and API Gateway throttling."

High concurrency optimization?

Answer:

"I use async endpoints, connection pooling, caching, load balancing, and horizontal scaling."

8. Embeddings

What are embeddings?

Answer:

"Embeddings are dense vector representations of text that capture semantic meaning."

Which embedding models?

Answer:

"I have primarily used Amazon Titan Embeddings."

You can also mention:

"Conceptually, I am familiar with OpenAI embeddings and sentence-transformer models."

Factors affecting embedding quality?

Answer:

"Embedding model quality, chunking strategy, preprocessing quality, document structure, and domain-specific terminology."

Can embeddings be regenerated?

Answer:

"Yes."

However, if a different embedding model is used, all documents should ideally be re-embedded for consistency."

9. LLM Integration

Integrating OpenAI or other LLM APIs?

Answer:

"I use SDKs or REST APIs, construct prompts, send requests, process responses, and integrate them into application workflows."

API failures and retries?

Answer:

"I implement retries with exponential backoff, timeout controls, circuit breakers, and fallback mechanisms."

Reduce token usage?

Answer:

"I optimize prompts, reduce unnecessary context, use Top-K retrieval, caching, and smaller models where appropriate."

Secure API keys?

Answer:

"I store them in AWS Secrets Manager, Parameter Store, or environment variables and never hardcode them."

10. Scenario-Based Questions

Design a policy chatbot using RAG.

Answer:

"I would ingest policy documents into S3, chunk them, generate embeddings, store them in Pinecone, retrieve relevant chunks, and use an LLM to generate grounded answers."

Insurance claim denial system.

Answer:

"I would build a RAG system using historical claim denials, policy documents, and compliance guidelines."

The model would retrieve similar denial cases and recommend corrective actions."

Users complain about irrelevant answers.

Answer:

"I would investigate document quality, chunking strategy, embedding model, retrieval scores, metadata filtering, and prompt construction."

Same prompt gives different answers.

Answer:

"I would check temperature settings, retrieval consistency, prompt changes, model version changes, and randomness parameters."

Design a system for millions of documents daily.

Answer:

"I would use S3 for storage, Lambda for event-driven ingestion, distributed processing, Titan Embeddings, Pinecone for vector storage, FastAPI services on ECS/EKS, caching layers, CloudWatch monitoring, and CI/CD pipelines."

The architecture would be horizontally scalable and fault tolerant."

Last-minute L2 Tip

If they ask "**How did you personally contribute?**", always bring the answer back to:

"I worked on document ingestion, chunking, embedding generation using Titan Embeddings, vector indexing in Pinecone/FAISS, LangChain/LangGraph orchestration, FastAPI APIs, prompt engineering, evaluation, and AWS deployment support."

That answer aligns perfectly with both projects on your resume and avoids getting trapped in areas like database schema design or infrastructure ownership that aren't explicitly on your resume.

Tell me all the real time production level questions that can be asked and solution / approach for those

This is where most L2 candidates fail.

The interviewer stops asking definitions and starts asking:

"Okay, this works in a demo. What happens in production?"

For your resume (Banking Agentic AI + Sanofi RAG), these are the most likely **production-level scenario questions**.

1. Retrieval Quality Suddenly Drops

Question

Your RAG system was working fine last month. Users are now getting irrelevant answers. What will you do?

Answer

"I would troubleshoot layer by layer.

First, I would verify whether new documents were correctly ingested.

Then I would check chunking quality and embedding generation.

Next, I would examine retrieval scores from Pinecone or FAISS.

I would compare retrieval performance against a golden dataset.

If retrieval is correct, I would inspect prompt construction and LLM outputs.

In production, most RAG failures occur in retrieval rather than generation."

2. Pinecone Returns Irrelevant Chunks

Question

Your vector search is returning unrelated documents.

Answer

"I would check whether the embedding model changed.

Then I would verify metadata filters.

I would inspect chunk sizes and overlap.

I would analyze similarity scores.

If required, I would introduce reranking models to improve retrieval quality."

3. Hallucinations Increased Suddenly

Question

Yesterday hallucination rate was 5%.

Today it is 25%.

Answer

"I would first verify retrieval quality.

Then I would inspect whether prompt templates changed.

I would compare retrieved context length and completeness.

I would verify if the model version was upgraded.

Finally, I would evaluate responses against a golden dataset to isolate the issue."

4. Pinecone Index Size Increased 10x

Question

Storage cost suddenly increased.

Answer

"I would check for duplicate ingestion.

Many times the same document gets indexed repeatedly.

I would use document IDs and versioning.

I would remove stale vectors and re-index only modified documents."

5. New Documents Are Not Searchable

Question

User uploads a document but cannot query it.

Answer

"I would verify the entire ingestion pipeline.

Check S3 upload.

Check Lambda trigger execution.

Verify chunk generation.

Verify embedding creation.

Confirm vectors are written to Pinecone.

Finally, verify namespace and metadata filters."

6. Lambda Timeout

Question

PDF size is 500MB.

Lambda keeps failing.

Answer

"I would move preprocessing to ECS or EC2.

Lambda is suitable for lightweight event-driven workloads.

Large document processing should be handled by containerized workers."

7. Millions Of Documents Daily

Question

How would you scale ingestion?

Answer

"I would implement a distributed ingestion architecture.

S3 uploads trigger SQS.

Worker services running on ECS process chunks.

Embeddings are generated in parallel.

Vectors are batch inserted into Pinecone.

This avoids Lambda bottlenecks."

8. LLM Cost Increased

Question

Bedrock cost doubled this month.

Answer

"I would first analyze token usage.

Reduce context size.

Optimize chunk retrieval.

Introduce caching.

Use smaller models for simpler tasks.

Reserve expensive models only for complex reasoning."

9. Slow Response Time

Question

Users complain that responses take 20 seconds.

Answer

"I would measure latency at every layer.

Retrieval latency.

Network latency.

Prompt construction.

LLM inference.

Database calls.

Then optimize the bottleneck rather than the entire system."

10. FastAPI Server Crash

Question

Thousands of users hit your API simultaneously.

Answer

"I would deploy multiple FastAPI instances behind a load balancer.

Use async endpoints.

Introduce autoscaling.

Add Redis caching.

Use ECS or Kubernetes for horizontal scaling."

11. One Agent Fails

Banking Project

Question

Credit Assessment Agent fails.

What happens?

Answer

"I would implement retries and fallback logic.

The Supervisor Agent should detect the failure.

It can either retry the task or proceed with partial information and flag the recommendation as incomplete."

12. MCP Server Down

Question

Customer Profile MCP Server is unavailable.

Answer

"I would use retries with exponential backoff.

If still unavailable, the agent should return a graceful error.

The workflow should continue wherever possible instead of failing completely."

13. Conflicting Agent Outputs

Question

Credit Agent says approve.

Risk Agent says reject.

Answer

"In our architecture, the Supervisor Agent consolidates outputs.

The final recommendation is generated based on all agent findings and supporting evidence.

Human review acts as the final decision layer."

14. Loan Policy Changes

Question

Bank updates underwriting policy.

Answer

"I would re-ingest only the updated documents.

Generate fresh embeddings.

Update Pinecone.

No model retraining is required because RAG retrieves the latest knowledge."

15. Same Query Different Answers

Question

Why?

Answer

"Possible reasons include:

- Temperature settings
 - Different retrieved chunks
 - Model updates
 - Prompt changes
 - Context window differences"
-

16. PII Data Exposure

Question

Customer SSN appears in output.

Answer

"I would implement PII masking before prompt generation.

Use role-based access controls.

Encrypt sensitive information.

Apply output validation before displaying results."

17. Large PDF Handling

Question

1000-page PDF.

Answer

"I would process pages in batches.

Perform OCR if required.

Create document-aware chunks.

Use parallel embedding generation.

Store metadata for traceability."

18. Table Split Across Pages**Question**

Common Banking Question

Answer

"I would use document intelligence tools such as Textract.

Before chunking, I would reconstruct the table structure.

The goal is to preserve logical relationships between rows and columns."

19. Chunking Strategy Failed**Question**

Users get incomplete answers.

Answer

"I would experiment with chunk size and overlap.

Sometimes chunks are too small and lose context.

Sometimes they are too large and retrieval becomes noisy.

I would evaluate different configurations using a golden dataset."

20. Model Upgrade**Question**

Claude 3.5 → Claude 4

Answer

"I would perform A/B testing.

Compare retrieval relevance.

Answer quality.

Latency.

Cost.

Hallucination rate.

Only after successful validation would I promote the model."

21. Production Monitoring

Question

What metrics would you monitor?

Answer

"I would monitor:

- Retrieval Precision
 - Retrieval Recall
 - Latency
 - Token Usage
 - Cost
 - Hallucination Rate
 - User Feedback
 - API Failures
 - Pinecone Query Latency
 - MCP Failure Rate"
-

22. Banking Project Hard Question

Question

Why Agentic AI instead of a single RAG pipeline?

Answer

"A single RAG pipeline is good for retrieving information.

Loan underwriting requires multiple independent analyses such as credit evaluation, financial analysis, risk assessment, policy validation, and recommendation generation.

Agentic AI enables specialized agents to handle each responsibility separately and collaborate to produce a more reliable decision."

23. Hardest Question You Might Get

Question

If you were redesigning this system today, what would you improve?

Answer

"I would introduce reranking models to improve retrieval quality.

I would add agent observability using LangSmith.

I would implement semantic caching.

I would add automated evaluation pipelines.

I would improve MCP resiliency and introduce more detailed monitoring dashboards.